Stairio Robot Prototype Reproduction-Integration Guide

This document consolidates the complete operating guide for the rail inspection trolley prototype development, covering the joint integration and debugging of the ESP32-CAM video transmission module (with Type-C download base) and the TB6612FNG driver. It applies to the full workflow — modular code refactoring, hardware wiring, resource slimming, and Web UI state interaction — carried out in the PlatformIO IDE environment.

1. Hardware Connection Topology and Power Strategy

The system adopts a "power and logic separated" supply architecture to avoid electrical interference from the high-load motors on the main control and video transmission link.

1.1 Power Management Module (MP1584)

The stable output of the MP1584 step-down module is the key to enabling the ESP32-CAM camera and Wi-Fi radio.

- **Input (IN+ / IN-)**: Connected to two 18650 Li-ion batteries in series (about 8.4V when fully charged, 7.4V nominal).
- **Output (OUT+ / OUT-)**: Using a multimeter in DC voltage mode, adjust the precision potentiometer on the module. **The no-load output voltage must be precisely calibrated and stabilized between 5.0V and 5.2V**.
- **Note**: Never connect it to the main board before the voltage has been calibrated to 5V.

1.2 ESP32-CAM Main Board and Download Base

- Insert the OV2640 camera's flexible flat cable into the socket on the main board (metal contacts facing the PCB on the underside), then press down the latch.
- Plug the ESP32-CAM into the CH340 programming base with the Type-C interface.
- **Power supply network**: Connect the calibrated 5V output of the MP1584 step-down module to the 5V pin of the programming base (or main board), while ensuring the entire system shares a common ground (GND).

1.3 TB6612FNG Motor Driver Pin Mapping

To avoid conflicts with the OV2640 camera data bus and clock signals, and to prevent abnormal physical pin levels during the ESP32 startup (strapping) phase, the system carries out careful pin

allocation.

TB6612 Port	Target Connection	Signal Description
VM	Battery pack positive (8.4V)	Direct motor power supply, bypassing the step-down module
VCC	Step-down module 5V output	Logic chip power supply
GND	Global common ground	Connected to battery negative and step-down module negative
STBY	3.3V or 5V (high level)	Driver enable. Must be pulled high, otherwise the motors lock up and won't spin
PWMA	ESP32 **GPIO 12**	Motor PWM speed control
AIN1	ESP32 **GPIO 13**	Motor direction control 1
AIN2	ESP32 **GPIO 14**	Motor direction control 2
AO1 / AO2	JGA25-370 motor	Power output channel

1.4 Sensor Merging and Hardware Wired-AND

To safely trigger the emergency stop without occupying dangerous system startup pins, the environmental sensors are consolidated and reused:

- **Input pin**: **GPIO 15** (`LIMIT_PIN`)
- **Infrared obstacle avoidance module**: Connect its `OUT` signal line directly to GPIO 15.
- **Micro limit switch**: Connect its **COM** terminal to GND, and merge its **NO (normally open)** terminal with the infrared OUT signal line, both feeding into GPIO 15.

(Principle: whenever any sensor is triggered, it produces a physical low level, triggering a unified falling-edge interrupt.)

---

2. Build Environment Setup and System Slimming

Use PlatformIO instead of the Arduino IDE, and disable irrelevant processes through macro definitions to maximize the system's available memory.

2.1 Build Configuration (platformio.ini)

platformio.ini

```
[env:esp32cam]
platform = espressif32@6.3.2
board = esp32cam
framework = arduino
monitor_speed = 115200
board_build.partitions = huge_app.csv
```

2.2 Core Resource Release (app_httpd.cpp)

The stock ESP32-CAM firmware carries AI vision components that consume significant memory. We forcibly disable the face detection and recognition features at the top of `app_httpd.cpp`:

app_httpd.cpp

```
// === Disable all the AI functions to save memory ===
#define CONFIG_ESP_FACE_DETECT_ENABLED 0
#define CONFIG_ESP_FACE_RECOGNITION_ENABLED 0
// ================================================
```

---

3. Modular Code Architecture

Through refactoring, the system decouples the low-level hardware from the top-level Web UI.

3.1 Physical Driver Layer (src/motor.h & src/motor.cpp)

motor.h

```
#ifndef MOTOR_H
#define MOTOR_H

#include <Arduino.h>

#define MOTOR_DIR1 13
#define MOTOR_DIR2 14
#define MOTOR_PWM 12
#define LIMIT_PIN 15

const int freq = 1000;
```

```
const int pwmChannel = 4;
const int pwmResolution = 8;

extern volatile bool emergency_stop;

void motor_init();
void car_forward(int speed);
void car_backward(int speed);
void car_stop();

#endif
```

It encapsulates the internal interrupt and implements instant speed control and power cut-off via `ledcWrite`.

motor.cpp

```
#include "motor.h"

volatile bool emergency_stop = false;

void IRAM_ATTR onLimitSwitchTriggered() {
    ledcWrite(pwmChannel, 0);
    emergency_stop = true;
}

void motor_init() {
    pinMode(MOTOR_DIR1, OUTPUT);
    pinMode(MOTOR_DIR2, OUTPUT);

    ledcSetup(pwmChannel, freq, pwmResolution);
    ledcAttachPin(MOTOR_PWM, pwmChannel);

    pinMode(LIMIT_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(LIMIT_PIN), onLimitSwitchTriggered, FALLING);

    car_stop();
}

void car_forward(int speed) {
    if (emergency_stop) return;
    digitalWrite(MOTOR_DIR1, HIGH);
    digitalWrite(MOTOR_DIR2, LOW);
    ledcWrite(pwmChannel, speed);
}
```

```
29
30  void car_backward(int speed) {
31      emergency_stop = false;
32      digitalWrite(MOTOR_DIR1, LOW);
33      digitalWrite(MOTOR_DIR2, HIGH);
34      ledcWrite(pwmChannel, speed);
35  }
36
37  void car_stop() {
38      digitalWrite(MOTOR_DIR1, LOW);
39      digitalWrite(MOTOR_DIR2, LOW);
40      ledcWrite(pwmChannel, 0);
41  }
```

3.2 Dynamic Frontend (src/motor_ui.h)

Using the JavaScript `Fetch API` and `setInterval`, it implements refresh-free control dispatch and real-time status polling (Ajax Polling), closing the loop on the UI side to display emergency stop alerts.

motor_ui.h

```
1   #ifndef MOTOR_UI_H
2   #define MOTOR_UI_H
3
4   static const char motor_html[] = R"rawliteral(
5   <!DOCTYPE html>
6   <html>
7   <head>
8       <meta charset="utf-8">
9       <meta name="viewport" content="width=device-width, initial-scale=1">
10      <title>Stairio Control</title>
11      <style>
12          body { font-family: Arial, sans-serif; text-align: center; margin-top:
    20px; background-color: #222; color: #fff; }
13          #status_banner { margin: 15px auto; padding: 12px; width: 80%; max-
    width: 300px; font-size: 18px; font-weight: bold; border-radius: 8px;
    background-color: #4CAF50; transition: background-color 0.3s; }
14          .slider-container { margin: 20px 0; }
15          input[type=range] { width: 70%; max-width: 300px; }
16          .btn-group { display: flex; flex-direction: column; align-items:
    center; gap: 15px; }
17          .btn { width: 120px; height: 50px; font-size: 18px; border-radius:
    8px; border: none; cursor: pointer; color: #fff; font-weight: bold; }
```

```
        .btn-fw { background-color: #4CAF50; }
        .btn-stop { background-color: #f44336; }
        .btn-bw { background-color: #2196F3; }
        .btn-clear { background-color: #FF9800; margin-top: 15px; }
        .btn:active { opacity: 0.7; }
    </style>
</head>
<body>
    <h2>Stairio Test</h2>
    <div id="status_banner">State: (Normal)</div>

    <div class="slider-container">
        <label for="speed">PWM duty cyle: <span id="speed_val">150</span>
</label><br><br>
        <input type="range" id="speed" min="0" max="255" value="150"
oninput="document.getElementById('speed_val').innerText=this.value">
    </div>

    <div class="btn-group">
        <button class="btn btn-fw"
onclick="sendCmd('forward')">Forward</button>
        <button class="btn btn-stop" onclick="sendCmd('stop')">Stop</button>
        <button class="btn btn-bw"
onclick="sendCmd('backward')">Backward</button>
        <button class="btn btn-clear" onclick="sendCmd('clear_estop')">Disable
Emergency</button>
    </div>

    <script>
        function sendCmd(action) {
            var speed = document.getElementById('speed').value;
            fetch('/motor?action=' + action + '&speed=' + speed);
            setTimeout(fetchStatus, 100);
        }

        function fetchStatus() {
            fetch('/motor?action=status')
                .then(response => response.json())
                .then(data => {
                    var banner = document.getElementById('status_banner');
                    if (data.emergency_stop) {
                        banner.style.backgroundColor = "#E53935";
                        banner.innerText = "State: Emergency Stop Enabled!";
                    } else {
                        banner.style.backgroundColor = "#4CAF50";
                        banner.innerText = "State: Normal";
                    }
```

```
                })
                .catch(err => console.log("Status fetch error"));
        }
        setInterval(fetchStatus, 1000);
    </script>
</body>
</html>
)rawliteral";

#endif
```

3.3 HTTP Routing Dispatch Center (src/app_httpd.cpp)

In the `motor_control_handler` at the bottom of `app_httpd.cpp`, the `status` action is intercepted and JSON data is returned; otherwise the corresponding motor command is executed, and the main web page is served when no parameters are present.

app_httpd.cpp

```
static esp_err_t motor_control_handler(httpd_req_t *req) {
    char buf[100];
    char action[32] = {0};
    char speed_str[32] = {0};
    int speed_val = 150;

    if (httpd_req_get_url_query_str(req, buf, sizeof(buf)) == ESP_OK) {
        if (httpd_query_key_value(buf, "action", action, sizeof(action)) == ESP_OK) {

            if (strcmp(action, "status") == 0) {
                char resp_str[64];
                snprintf(resp_str, sizeof(resp_str), "{\"emergency_stop\": %s}",
                         emergency_stop ? "true" : "false");
                httpd_resp_set_type(req, "application/json");
                httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
                return httpd_resp_send(req, resp_str, strlen(resp_str));
            }

            if (httpd_query_key_value(buf, "speed", speed_str, sizeof(speed_str)) == ESP_OK) {
                speed_val = atoi(speed_str);
            }

            if (strcmp(action, "forward") == 0) car_forward(speed_val);
            else if (strcmp(action, "backward") == 0) car_backward(speed_val);
```

```
            else if (strcmp(action, "stop") == 0) car_stop();
            else if (strcmp(action, "clear_estop") == 0) emergency_stop =
    false;

            httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
            return httpd_resp_send(req, NULL, 0);
        }
    }

    httpd_resp_set_type(req, "text/html");
    return httpd_resp_send(req, motor_html, strlen(motor_html));
}
```

Finally, mount this route in the `startCameraServer()` function:

app_httpd.cpp

```
        httpd_uri_t motor_uri = {
            .uri       = "/motor",
            .method    = HTTP_GET,
            .handler   = motor_control_handler,
            .user_ctx  = NULL
        };
        httpd_register_uri_handler(camera_httpd, &motor_uri);
```

---

4. Integration Testing and Troubleshooting

4.1 Compile and Power On

- First unplug the 3.3/5V interface on the ESP32.
- Confirm the build cache has been cleaned (Clean), then click Upload.
- Press the RST button on the base for a hard reset, and watch the serial output to make sure `Camera init failed with error 0xffffffff` does not appear (if it does, the cause is insufficient supply current, a loose ribbon cable, or the 3.3/5V interface not being unplugged).

4.2 Concurrent Web Testing

1. **Start the video stream**: In the browser, visit `http://<IP address>` to view the real-time video monitoring.
2. **Open the console and verify emergency stop**: Visit `http://<IP address>/motor` .

（注：部分内容由豆包工作 AI 生成）